

COURSE CODE/TITLE: CPE212 - Automating Server Management	SECTION: CPE31S2
DATE PERFORMED: 08/25/2026	DATE SUBMITTED: 08/25/2026
NAME: Christopher T. Fegalan	INSTRUCTOR: Engr. Jimlord Quejado

ACTIVITY NO. 4.1 ANSIBLE BASICS

1. PROCEDURE OUTPUT

Server1	192.168.0.3
Server2	192.168.0.4

TASK 1:

```
user123@workstation:~/ansible$ ansible all -i hosts.ini -m apt -a update_cache=true
192.168.0.104 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "msg": "Failed to lock apt for exclusive operation: Failed to lock directory /var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)"
}
192.168.0.103 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "msg": "Failed to lock apt for exclusive operation: Failed to lock directory /var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)"
}
```

What is the result of the command? Is it successful? **No**

```
user123@workstation:~/ansible$ ansible all -i hosts.ini -m apt -a update_cache=true --become --ask-become-pass
BECOME password:
192.168.0.103 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787618179,
  "cache_updated": true,
  "changed": true
}
192.168.0.104 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787618181,
  "cache_updated": true,
  "changed": true
}
```

Server1_Ubuntu_Linux [Running] - Oracle VirtualBox

```

user123@server1: ~
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

user123@server1:~$ whoami
user123
user123@server1:~$ which vim
user123@server1:~$ ansible all -i inventory.yaml -m command -a "apt-get install
-f -y" --become --ask-become-pass
BECOME password:
[WARNING]: Unable to parse /home/user123/inventory.yaml as an inventory source
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that
the implicit localhost does not match 'all'
user123@server1:~$ which vim
/usr/bin/vim
user123@server1:~$ apt search vim-nox
Sorting... Done
Full Text Search... Done
vim-nox/noble 2:9.1.0016-1ubuntu7 amd64
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/noble 2:9.1.0016-1ubuntu7 amd64 [residual-config]
  Vi IMproved - enhanced vi editor - compact version

user123@server1:~$

```

Server2_Ubuntu [Running] - Oracle VirtualBox

```

user123@server2: ~
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 351 bytes 33906 (33.9 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 351 bytes 33906 (33.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

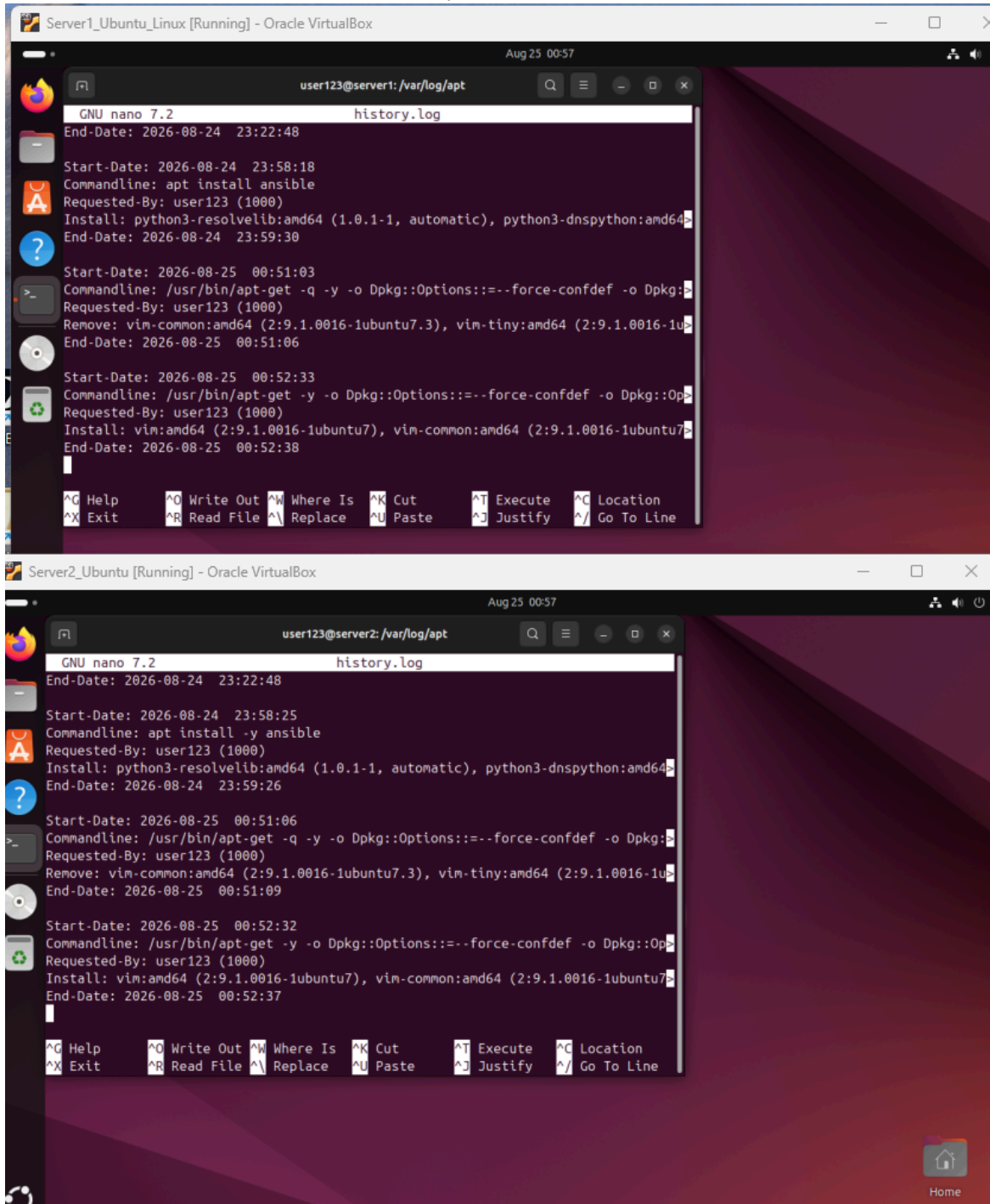
user123@server2:~$ which vim
user123@server2:~$ which vim
/usr/bin/vim
user123@server2:~$ apt search vim-nox
Sorting... Done
Full Text Search... Done
vim-nox/noble 2:9.1.0016-1ubuntu7 amd64
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/noble 2:9.1.0016-1ubuntu7 amd64 [residual-config]
  Vi IMproved - enhanced vi editor - compact version

user123@server2:~$

```

Verify that you have installed the package in the remote servers. Issue the command `vim` and the command `apt search vim-nox` respectively. Was the command successful? **Yes**



The image shows two terminal windows from Oracle VM VirtualBox, each displaying the `history.log` file in the `/var/log/apt` directory. The windows are titled `Server1_Ubuntu_Linux [Running] - Oracle VirtualBox` and `Server2_Ubuntu [Running] - Oracle VirtualBox`. Both logs show the installation of `vim` on 2026-08-25 at 00:52:32. The logs also show the removal of `vim-common` and `vim-tiny` on 2026-08-25 at 00:51:06. The terminal windows have a dark purple background with a sidebar on the left containing icons for applications, files, and settings. The bottom of the windows shows a status bar with system icons and a 'Home' button.

```

user123@server1: /var/log/apt
GNU nano 7.2 history.log
End-Date: 2026-08-24 23:22:48
Start-Date: 2026-08-24 23:58:18
Commandline: apt install ansible
Requested-By: user123 (1000)
Install: python3-resolvelib:amd64 (1.0.1-1, automatic), python3-dnspython:amd64
End-Date: 2026-08-24 23:59:30
Start-Date: 2026-08-25 00:51:03
Commandline: /usr/bin/apt-get -q -y -o Dpkg::Options::=-force-confdef -o Dpkg:
Requested-By: user123 (1000)
Remove: vim-common:amd64 (2:9.1.0016-1ubuntu7.3), vim-tiny:amd64 (2:9.1.0016-1u
End-Date: 2026-08-25 00:51:06
Start-Date: 2026-08-25 00:52:33
Commandline: /usr/bin/apt-get -y -o Dpkg::Options::=-force-confdef -o Dpkg::Op
Requested-By: user123 (1000)
Install: vim:amd64 (2:9.1.0016-1ubuntu7), vim-common:amd64 (2:9.1.0016-1ubuntu7
End-Date: 2026-08-25 00:52:38
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^M Replace ^U Paste ^J Justify ^_ Go To Line

user123@server2: /var/log/apt
GNU nano 7.2 history.log
End-Date: 2026-08-24 23:22:48
Start-Date: 2026-08-24 23:58:25
Commandline: apt install -y ansible
Requested-By: user123 (1000)
Install: python3-resolvelib:amd64 (1.0.1-1, automatic), python3-dnspython:amd64
End-Date: 2026-08-24 23:59:26
Start-Date: 2026-08-25 00:51:06
Commandline: /usr/bin/apt-get -q -y -o Dpkg::Options::=-force-confdef -o Dpkg:
Requested-By: user123 (1000)
Remove: vim-common:amd64 (2:9.1.0016-1ubuntu7.3), vim-tiny:amd64 (2:9.1.0016-1u
End-Date: 2026-08-25 00:51:09
Start-Date: 2026-08-25 00:52:32
Commandline: /usr/bin/apt-get -y -o Dpkg::Options::=-force-confdef -o Dpkg::Op
Requested-By: user123 (1000)
Install: vim:amd64 (2:9.1.0016-1ubuntu7), vim-common:amd64 (2:9.1.0016-1ubuntu7
End-Date: 2026-08-25 00:52:37
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^M Replace ^U Paste ^J Justify ^_ Go To Line
  
```

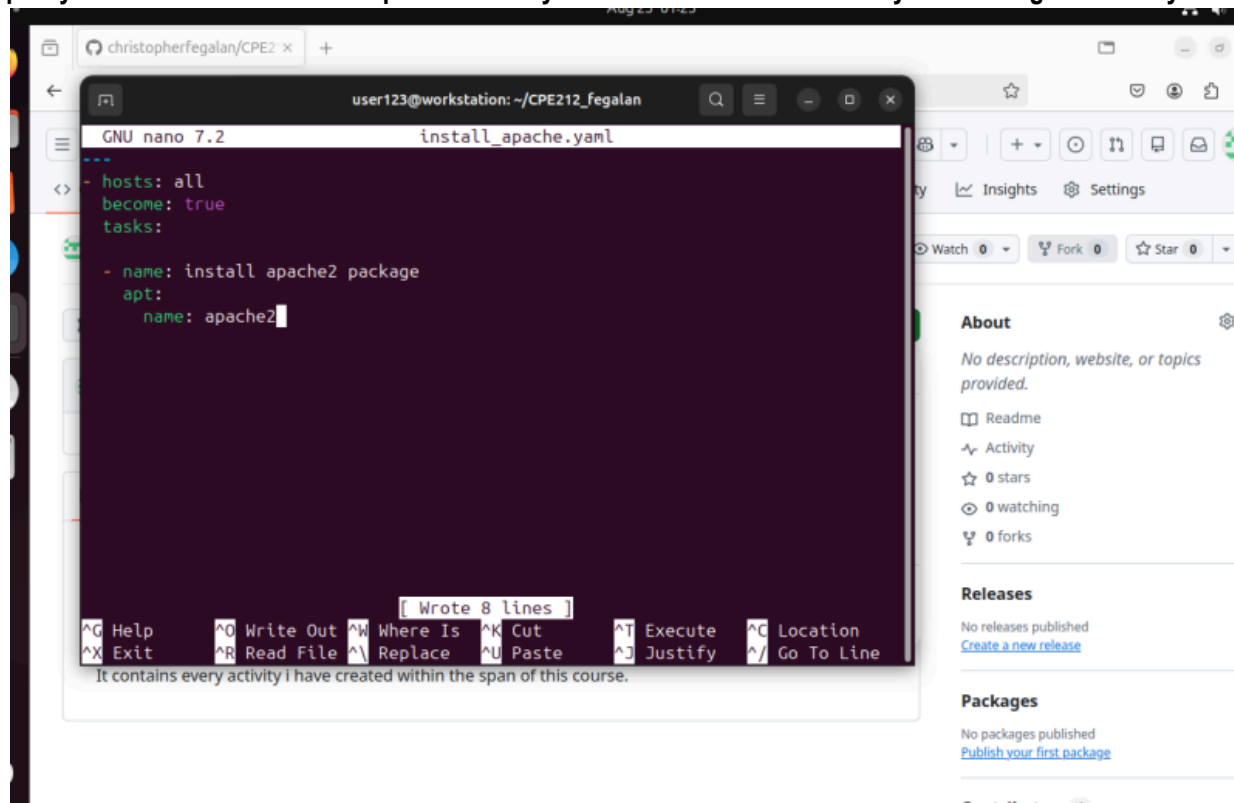
Check the logs in the servers using the following commands: `cd /var/log`. After this, issue the command `ls`, go to the folder `apt` and open `history.log`. Describe what you see in the `history.log`.

- It shows the commands I previously used through ansible using my workstation vm.

Issue the command: `ansible all -m apt -a name=snapd --become --ask-become-pass`

```
user123@workstation:~/ansible$ which vim
user123@workstation:~/ansible$ ansible all -i inventory.yaml -m apt -a name=snapd --become --ask-become-pass
BECOME password:
ubuntu2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787619109,
  "cache_updated": false,
  "changed": false
}
ubuntu1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787619107,
  "cache_updated": false,
  "changed": false
}
```

Can you describe the result of this command? Is it a success? Did it change anything in the remote servers? **It was a success for both vms. But from what I can tell it didn't really change anything. I'm pretty sure this was because snapd was already on both servers so it basically did nothing but identify it.**



```

user123@workstation: ~/CPE212_fegalan$ nano install_apache.yml
user123@workstation:~/CPE212_fegalan$ ansible-playbook --ask-become-pass install
_apache.yml
BECOME password:

PLAY [all] *****

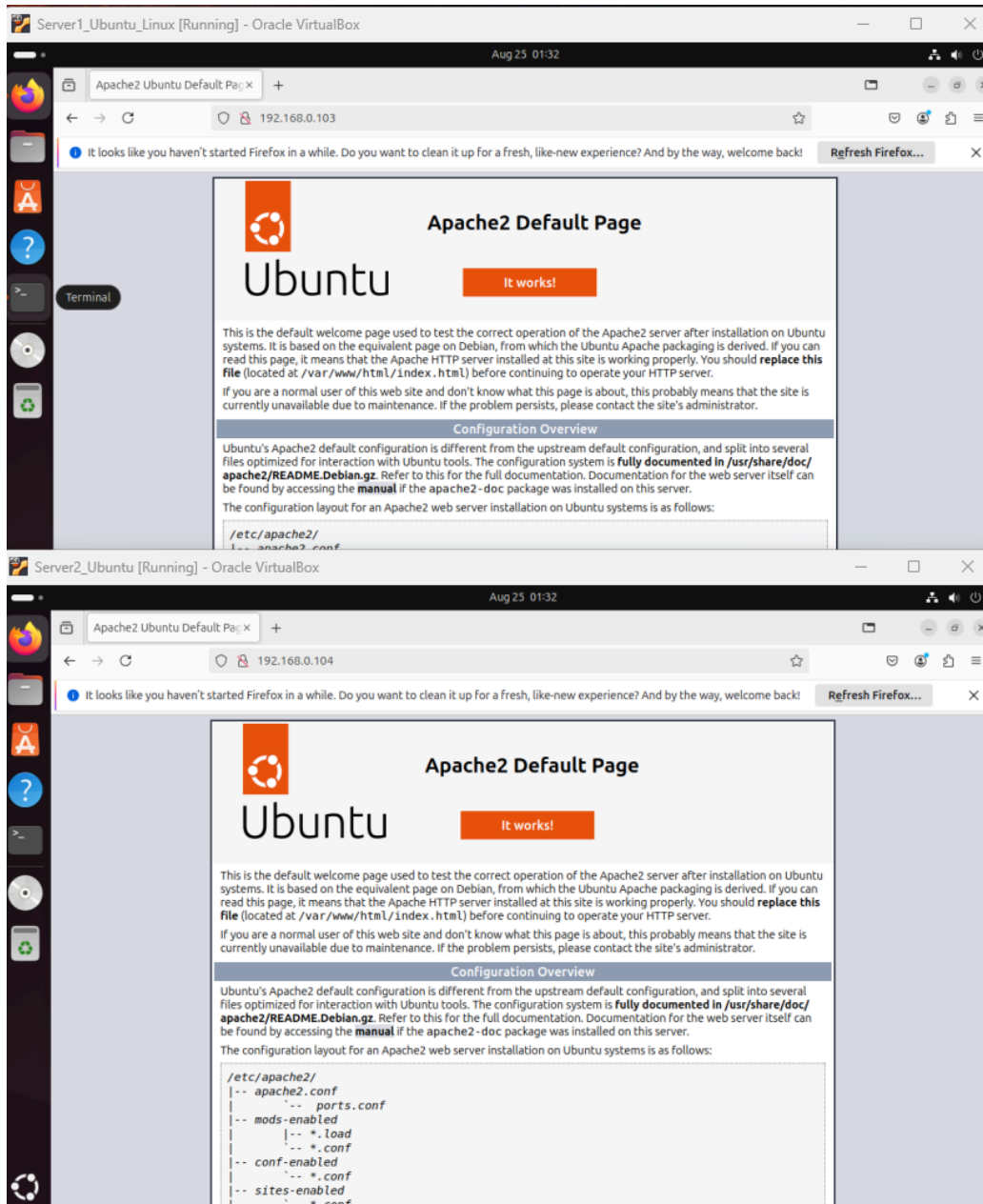
TASK [Gathering Facts] *****
ok: [192.168.0.103]
ok: [192.168.0.104]

TASK [install apache2 package] *****
changed: [192.168.0.103]
changed: [192.168.0.104]

PLAY RECAP *****
192.168.0.103      : ok=2    changed=1    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
192.168.0.104      : ok=2    changed=1    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
user123@workstation:~/CPE212_fegalan$

```

Run the yml file using the command: `ansible-playbook --ask-become-pass install_apache.yml`. Describe the result of this command. **It was able to install the apache2 packages to both the vms. The changed status shows that it was able to safely change or install the apache webservice.**



```

user123@workstation: ~/CP
GNU nano 7.2 install_apac

--
hosts: all
become: true
tasks:

- name: install apache2 package
  apt:
    name: invalidapache_23232zzzz

install_apache.yaml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.0.104]
ok: [192.168.0.103]

TASK [install apache2 package] *****
fatal: [192.168.0.104]: FAILED! => {"changed": false, "msg": "No package matching 'invalidapache_23232zzzz' is available"}
fatal: [192.168.0.103]: FAILED! => {"changed": false, "msg": "No package matching 'invalidapache_23232zzzz' is available"}

PLAY RECAP *****
192.168.0.103      : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.0.104      : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0

```

Try to edit the install_apache.yml and change the name of the package to any name that will not be recognized. What is the output? **Fatal**

```

apache2.yam
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.0.103]
ok: [192.168.0.104]

TASK [update repository index] *****
changed: [192.168.0.104]
changed: [192.168.0.103]

TASK [install apache2 package] *****
ok: [192.168.0.104]
ok: [192.168.0.103]

PLAY RECAP *****
192.168.0.103      : ok=3    changed=1    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
192.168.0.104      : ok=3    changed=1    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0

```

Run the playbook and describe the output. Did the new command change anything on the remote servers? The **playbook successfully updated the package repository on both servers, while Apache2 was verified as already installed.**

```

user123@workstation: ~/CPE212_fegalan
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.0.103]
ok: [192.168.0.104]

TASK [update repository index] *****
changed: [192.168.0.104]
changed: [192.168.0.103]

TASK [install apache2 package] *****
ok: [192.168.0.104]
ok: [192.168.0.103]

TASK [add PHP support for apache] *****
changed: [192.168.0.104]
changed: [192.168.0.103]

PLAY RECAP *****
192.168.0.103      : ok=4    changed=2    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
192.168.0.104      : ok=4    changed=2    unreachable=0    failed=0    s

```

Run the playbook and describe the output. Did the new command change anything on the remote servers? The **new command added the php support for apache on both servers and also updated the repository index.**

Finally, make sure that we are in sync with GitHub. Provide the link of your GitHub repository

https://github.com/christopherfegalan/CPE212_fegalan

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Answer the following:

1. What is the importance of using a playbook?

- Compared to doing things by ad hoc. Playbook allows for the automation of tasks and is not restricted by the one and done rule of ad hoc commands. Playbooks are structured, and are easier to understand compared to ad hoc since it is written in a very readable manner. Running a playbook multiple times also doesn't break anything since they have these statuses where changes are only made when a part of the system is out of sync.

2. Summarize what we have done on this activity.

- The first few steps of this activity involved executing ad-hoc commands directly through the terminal to perform various tasks, such as installing vim and snap. This allowed me to observe and compare command outputs, seeing firsthand how Ansible manages remote VMs by reporting their state whether a task succeeded, made a modification, or failed. Later, I transitioned to using a playbook to run multiple tasks sequentially. This introduced a structured, repeatable way to handle administration and save tasks into a single file. Working with playbooks showed me the concept of idempotency, which shows that a playbook can be run repeatedly without breaking everything, as changes are only applied when a node is out of sync or missing required configurations. Finally, I managed the install_apache.yaml file using Git version control by cloning the repository, staging the changes, committing them, and pushing the updated code back to GitHub.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Debugging

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: It was used to debug problems that weren't addressed in this activity documentation. Mostly problems with installing vim because of version incompatibility.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.